Lab3 | РБПО

| Горбачева Маргарита Валерьевна | 24КНТ4

Задание:

Обязательно используйте LLM для генерации C++-кода. Тема приложения свободная. В промпте потребуйте: приложение с вводом из файла или stdin; минимум две намеренные ошибки безопасности разных типов, включая ошибку памяти; минимум одну реально используемую стороннюю зависимость фиксированной версии с известной CVE. Приложение должно собираться и выполнять свою основную функцию. Сохраните название LLM, промпт и исходный сгенерированный код. Ошибки, версии зависимостей и CVE, предложенные LLM, проверьте самостоятельно. Исходную дефектную версию сохраните отдельно от исправленной.

SAST. Сохранить команды, версии инструментов и отчёты. Для уникальных находок указать место,причину, CWE, статус TP/FP и обоснование. Отметить пропущенные намеренные ошибки. Исправить подтвержденные дефекты и повторить анализ.
SCA. Зафиксировать состав и версии зависимостей, получить SBOM. Разобрать CVE и их применимость к приложению. Обновить уязвимую зависимость, проверить сборку и поведение, повторить SCA. Для двух инструментов сравнить результаты.
Во всех этапах для каждой подтвержденной проблемы найти материал **OWASP** и запись **БДУ ФСТЭК**: привести ссылки и объяснить связь. Для CWE и сбоя ищется аналогичный случай, для CVE - запись о той же уязвимости. Если соответствие не найдено, описать поиск; идентификаторы не выдумывать.

Необходимо сдать:

- Исходная и исправленная версии проекта, инструкция сборки, промпт и название LLM, команды и отчёты инструментов, краткая таблица разбора с OWASP/БДУ. Для SCA добавить SBOM.

Предполагаемый балл выполненной работы - 8.

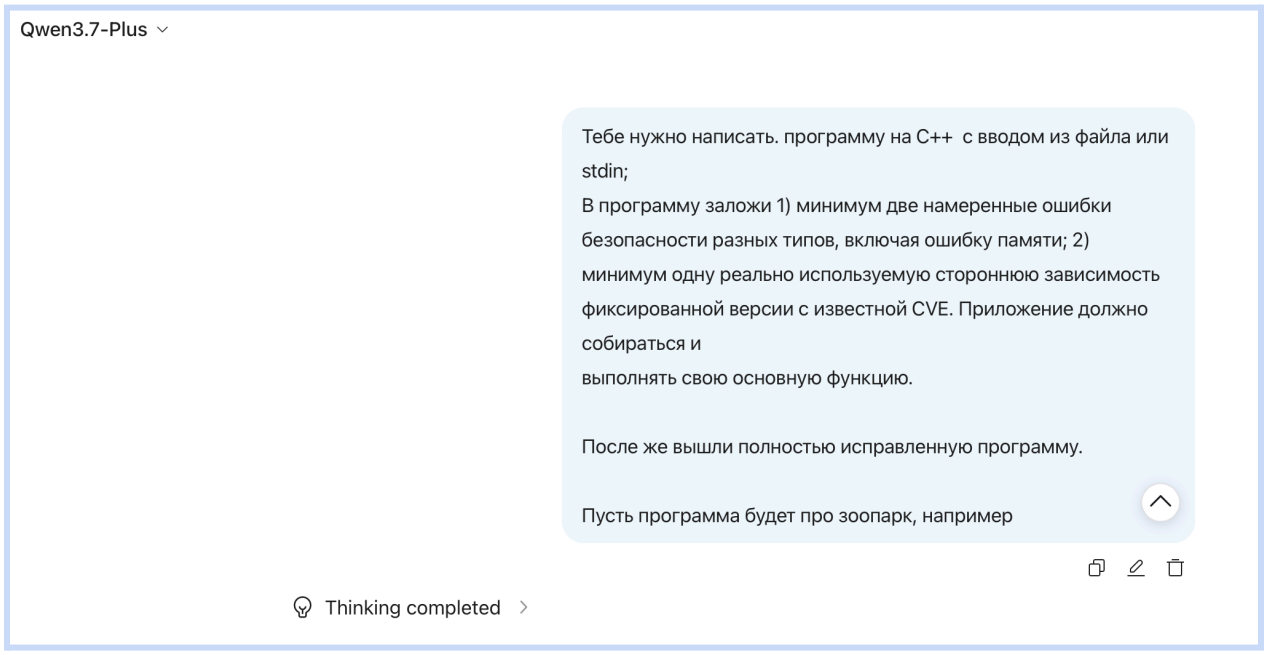
Структура работы:

1. Программа на C++ с намеренными уязвимостями в коде `vulnerableZoo.cpp`
2. Исправленная программа на C++ `okZoo.cpp`
3. Инструкция сборки
4. Промпт и название использованной для генерации кода LLM
5. Команды и отчеты инструментов

6. Краткая таблица разбора с OWASP/БДУ ФСТЭК

Для генерации кода была использована моя идея зоопарка и код от Qwen

Промпт:



Запуск программы:

1. Запуск уязвимой версии

```
g++ -O0 -g -o vulnerablezoo vulnerableZoo.cpp -lz
./vulnerablezoo
```

2. Запуск исправленной версии

```
g++ -O0 -g -o okzoo okZoo.cpp -lz
./okzoo
```

Затем можно вводить животных, например:

```
Penguin, Trololo, 5
```

Или же можно вводить из файла:

```
./vulnerablezoo input.txt
./okzoo input.txt
```

Важно: перед тем, как проверять файлы на уязвимости, необходимо установить пакеты для работы с SAST, SCA

```
brew install cppcheck clang-tidy flawfinder syft trivy

brew install llvm
```

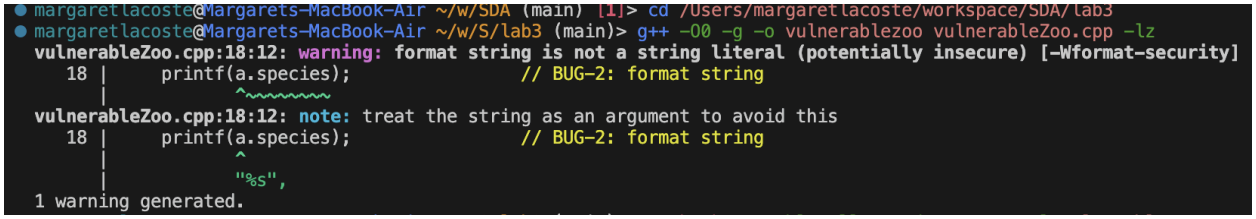
Использованные анализаторы и инструменты:

SAST - cppcheck, Clang-tidy, flawfinder

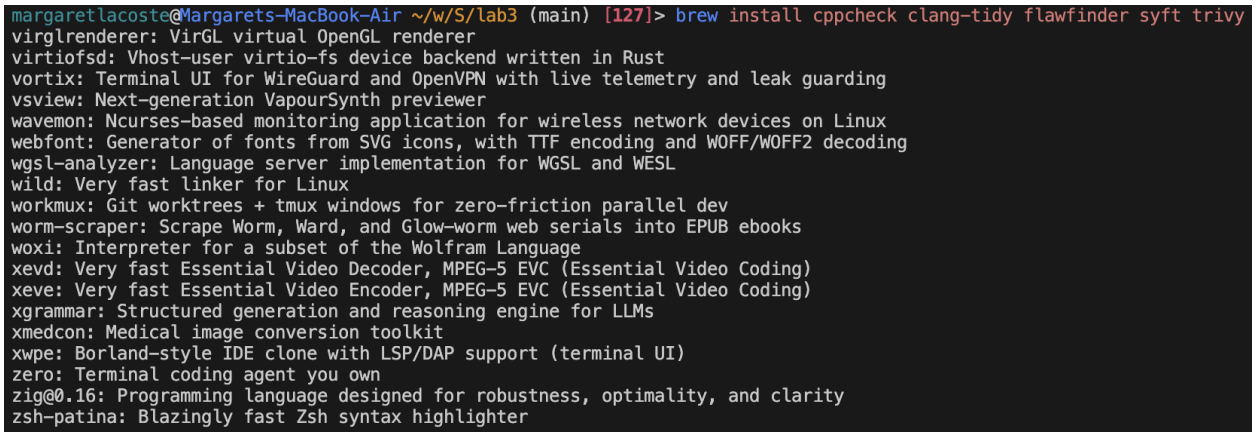
SCA - Syft (SBOM в CycloneDX), Trivy (поиск CVE)

Ход работы:

1) Запускаем наше приложение:



2) Настраиваем библиотеки



```
● margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main) [1]> brew install llvm
  ==> Downloading bottle manifests
  ✔ Bottle Manifest llvm (23.1.3_2)
  ==> Would install 1 formula:
  llvm 23.1.3_2
  ==> Would install 4 dependencies for llvm:
  lz4
  xz
  zstd
  z3
  ==> Do you want to proceed with the installation? [y/n]
  Invalid input. Please press 'y' to proceed, or 'n' to abort.
  ==> Fetching downloads for: llvm
  ✔ Bottle Manifest lz4 (1.10.0)
  ✔ Bottle Manifest z3 (5.3.1)
  ✔ Bottle Manifest zstd (1.5.7_1)
  ✔ Bottle Manifest xz (5.8.4)
  ✔ Bottle lz4 (1.10.0)
  ✔ Bottle zstd (1.5.7_1)
  ✔ Bottle xz (5.8.4)
```

3) Еще раз запустим приложение после установк необходимых пакетов

```
● margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> g++ -O0 -g -o vulnerable_zoo vulnerableZoo.cpp -lz
  vulnerableZoo.cpp:18:12: warning: format string is not a string literal (potentially insecure) [-Wformat-security]
     18 |     printf(a.species);               // BUG-2: format string
        |            ^~~~~~~~~
  vulnerableZoo.cpp:18:12: note: treat the string as an argument to avoid this
     18 |     printf(a.species);               // BUG-2: format string
        |            ^
        |            "%s",
  1 warning generated.
```

```
● margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> echo "Leo,Panthera,5" | ./vulnerable_zoo
  Panthera age=5
```

4) У нас почему-то не находился cppcheck, повторяем процедуры установки

```
⊙ margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main) [127]> cppcheck --enable=all --std=c++17 --xml vulnerableZoo.cpp 2> cppcheck.xml
  fish: Unknown command: cppcheck
⊙ margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main) [127]> brew list | grep cppcheck
● margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main) [0|1]> brew install cppcheck flawfinder syft trivy
  ==> Downloading bottle manifests
  ✔ Bottle Manifest trivy (0.76.0)                                                    Downloaded   12.8KB/ 12.8KB
  ✔ Bottle Manifest syft (1.54.1)                                                     Downloaded   12.8KB/ 12.8KB
  ✔ Bottle Manifest cppcheck (2.22.0)                                                 Downloaded   19.4KB/ 19.4KB
  ✔ Bottle Manifest flawfinder (2.0.20)                                               Downloaded   59.7KB/ 59.7KB
  ==> Would install 4 formulae:
  cppcheck    2.22.0
  flawfinder  2.0.20
  syft        1.54.1
  trivy       0.76.0
  ==> Would install 1 dependency for cppcheck:
  tinyxml2
  ==> Would install 6 dependencies for flawfinder:
  ca-certificates
  mpdecimal
  openssl@4
  readline
  sqlite
  python@3.15
  ==> Do you want to proceed with the installation? [y/n]
  ==> Fetching downloads for: cppcheck, flawfinder, syft and trivy
```

5) Проверяем, все хорошо

```
margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> brew list | grep -E "cppcheck|flawfinder|syft|trivy"
cppcheck
flawfinder
syft
trivy
margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> which cppcheck
/opt/homebrew/bin/cppcheck
margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> fish_add_path /opt/homebrew/bin /opt/homebrew/sbin
set -U fish_user_paths /opt/homebrew/bin /opt/homebrew/sbin
margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> which cppcheck
/opt/homebrew/bin/cppcheck
margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> cppcheck --version
                                                         flawfinder --version
                                                         syft version
                                                         trivy --version
                                                         clang-tidy --version

Cppcheck 2.22.0
2.0.20
Application:   syft
Version:       1.54.1
BuildDate:     2026-10-06T14:36:33Z
GitCommit:     Homebrew
GitDescription: [not provided]
Platform:      darwin/arm64
GoVersion:     go1.27.1
Compiler:      gc
```

6) Запускаем первый SAST анализатор (Cppcheck)

```
margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> cppcheck --enable=all --std=c++17 --xml vulnerableZoo.cpp 2> cppcheck.xml
Checking vulnerableZoo.cpp ...
margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> cat cppcheck.xml
<?xml version="1.0" encoding="UTF-8"?>
<results version="2">
    <cppcheck version="2.22.0"/>
    <errors>
        <error id="missingIncludeSystem" severity="information" msg="Include file: &lt;iostream&gt; not found. Please note: Standard library headers do no
t need to be provided to get proper results." verbose="Include file: &lt;iostream&gt; not found. Please note: Standard library headers do not need to be p
rovided to get proper results.">
            <location file="vulnerableZoo.cpp" line="1" column="2"/>
        </error>
        <error id="missingIncludeSystem" severity="information" msg="Include file: &lt;fstream&gt; not found. Please note: Standard library headers do not
 need to be provided to get proper results." verbose="Include file: &lt;fstream&gt; not found. Please note: Standard library headers do not need to be pro
vided to get proper results.">
            <location file="vulnerableZoo.cpp" line="2" column="2"/>
        </error>
```

На этом этапе формируется cppcheck.xml и report_cppcheck/index.html

lab3 > cppcheck.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<results version="2">
    <cppcheck version="2.22.0"/>
    <errors>
        <error id="missingIncludeSystem" severity="information" msg="Cannot find all the include files (use --check-config for details)" verbose="Cannot find all the include files (use --check-config for details)">
            <location file="vulnerableZoo.cpp" line="1" column="2"/>
        </error>
        <error id="missingIncludeSystem" severity="information" msg="Cannot find all the include files (use --check-config for details)" verbose="Cannot find all the include files (use --check-config for details)">
            <location file="vulnerableZoo.cpp" line="2" column="2"/>
        </error>
        <error id="missingIncludeSystem" severity="information" msg="Cannot find all the include files (use --check-config for details)" verbose="Cannot find all the include files (use --check-config for details)">
            <location file="vulnerableZoo.cpp" line="3" column="2"/>
        </error>
        <error id="missingIncludeSystem" severity="information" msg="Cannot find all the include files (use --check-config for details)" verbose="Cannot find all the include files (use --check-config for details)">
            <location file="vulnerableZoo.cpp" line="4" column="2"/>
        </error>
        <error id="missingIncludeSystem" severity="information" msg="Cannot find all the include files (use --check-config for details)" verbose="Cannot find all the include files (use --check-config for details)">
            <location file="vulnerableZoo.cpp" line="5" column="2"/>
        </error>
        <error id="missingIncludeSystem" severity="information" msg="Cannot find all the include files (use --check-config for details)" verbose="Cannot find all the include files (use --check-config for details)">
            <location file="vulnerableZoo.cpp" line="6" column="2"/>
```

lab3 > report_cppcheck > index.html > ...

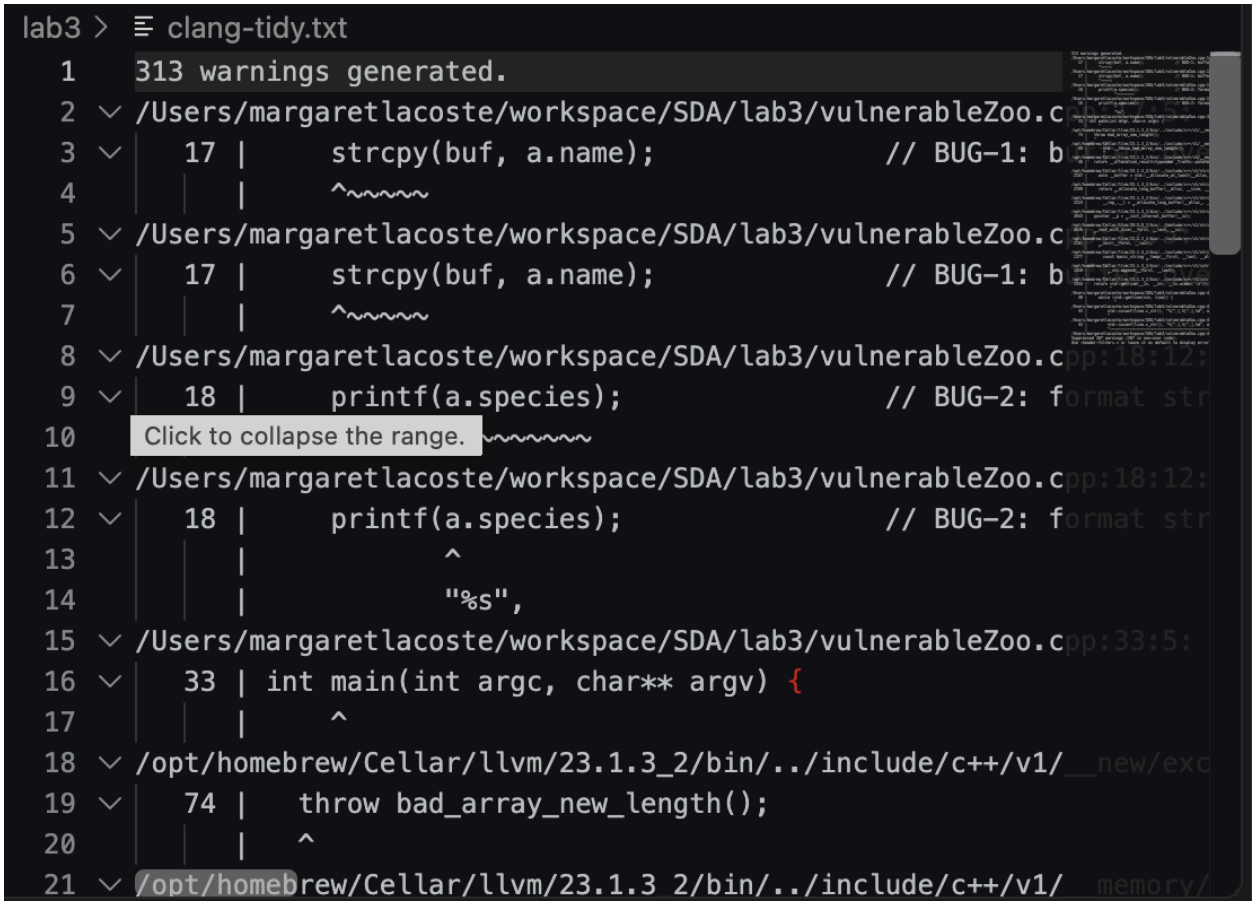
```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Cppcheck - HTML report - Cppcheck</title>
    <link rel="stylesheet" href="style.css">
    <style>

    </style>
    <script>
      function getStyle(el, styleProp) {
        var y;

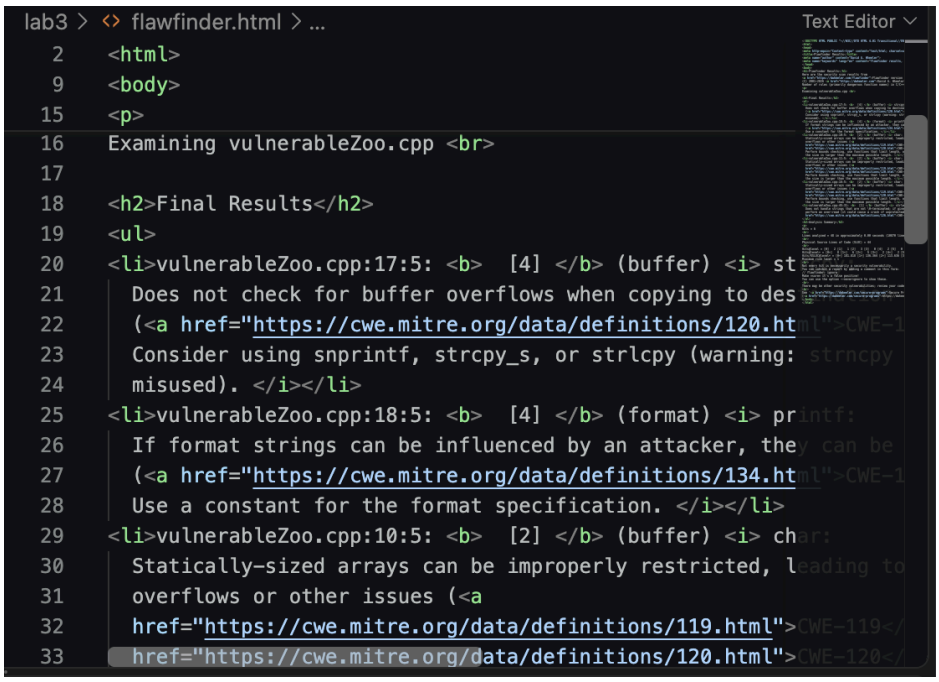
        if (el.currentStyle) {
          y = el.currentStyle[styleProp];
        } else if (window.getComputedStyle) {
          y = document.defaultView.getComputedStyle(el, nu
        }

        return y;
```

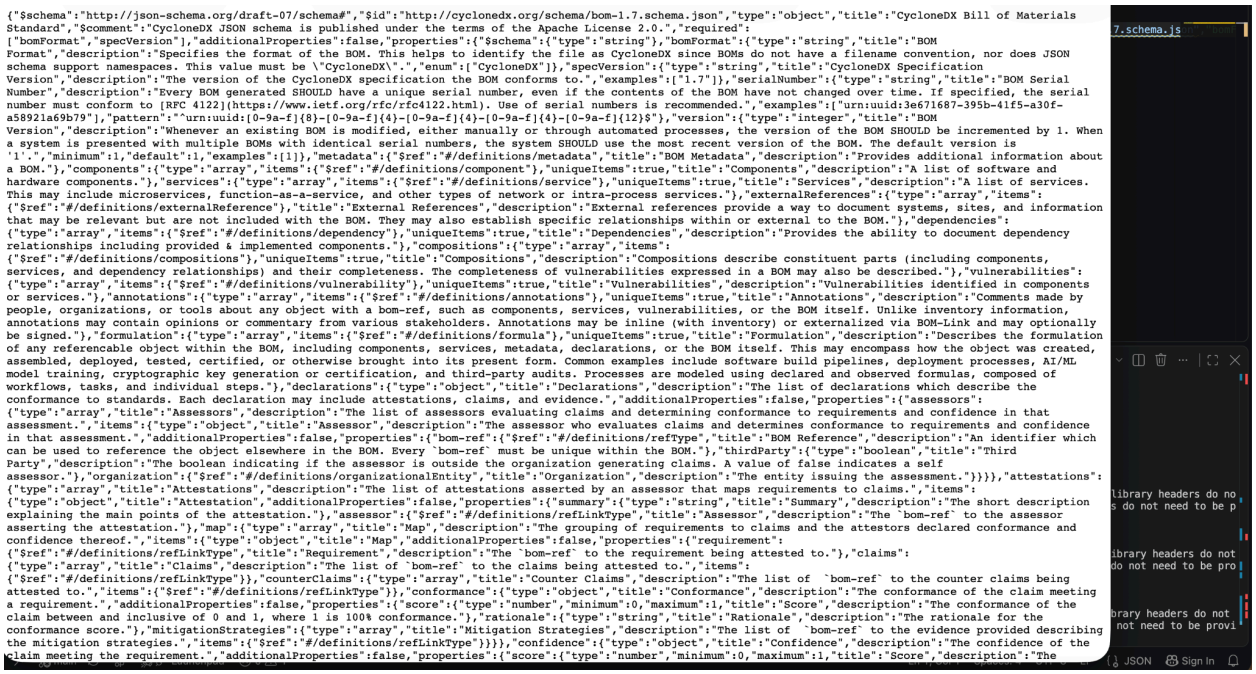
7) Далее запускаем второй SAST анализатор - Clang-Tidy



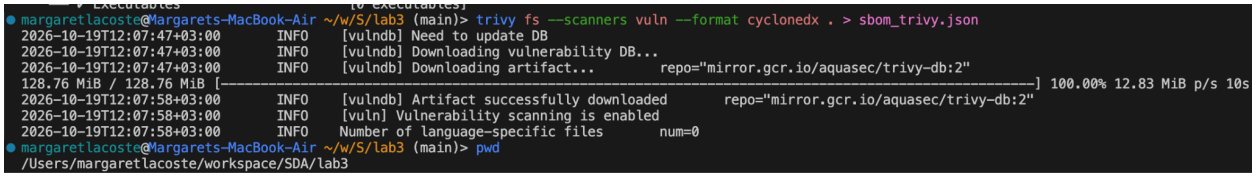
8) И теперь третий SAST анализатор - Flawfinder



9) А теперь переходим к SCA Syft (SBOM в CycloneDX)



10) И Trivy (поиск CVE)

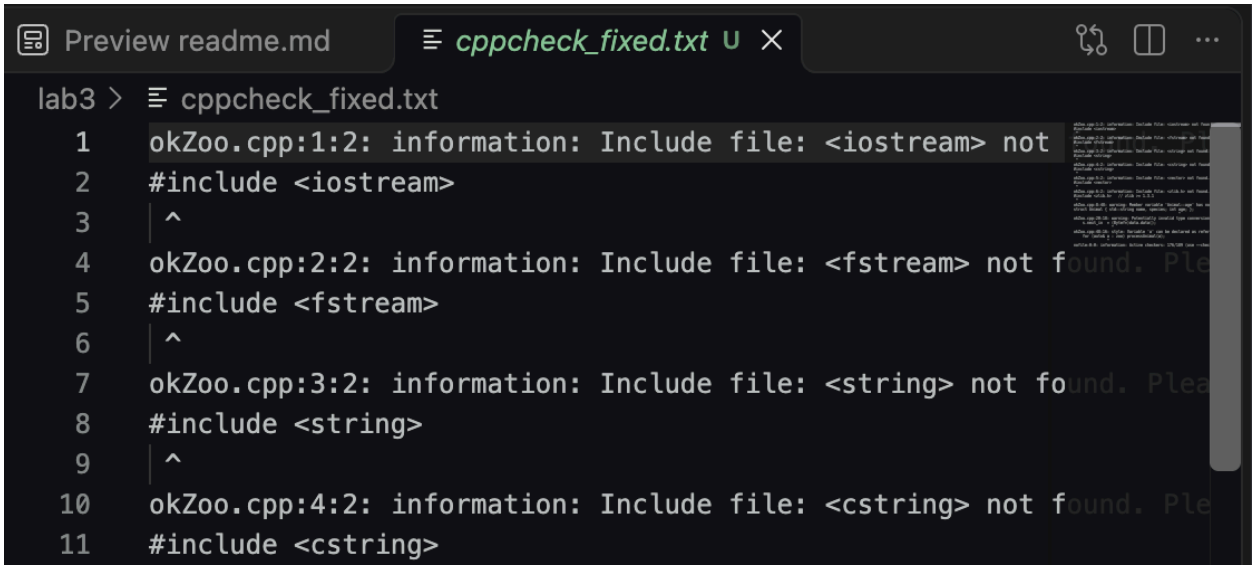




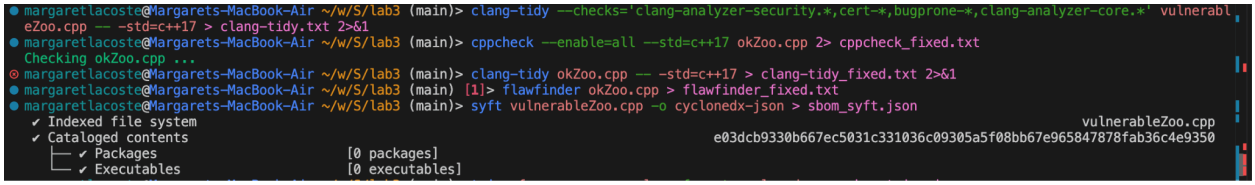
После всех этих штук мы запускаем SASTы на исправленной версии кода

(cppcheck_fixed.txt, slang-tidy_fixed.txt, flawfinder_fixed.txt)

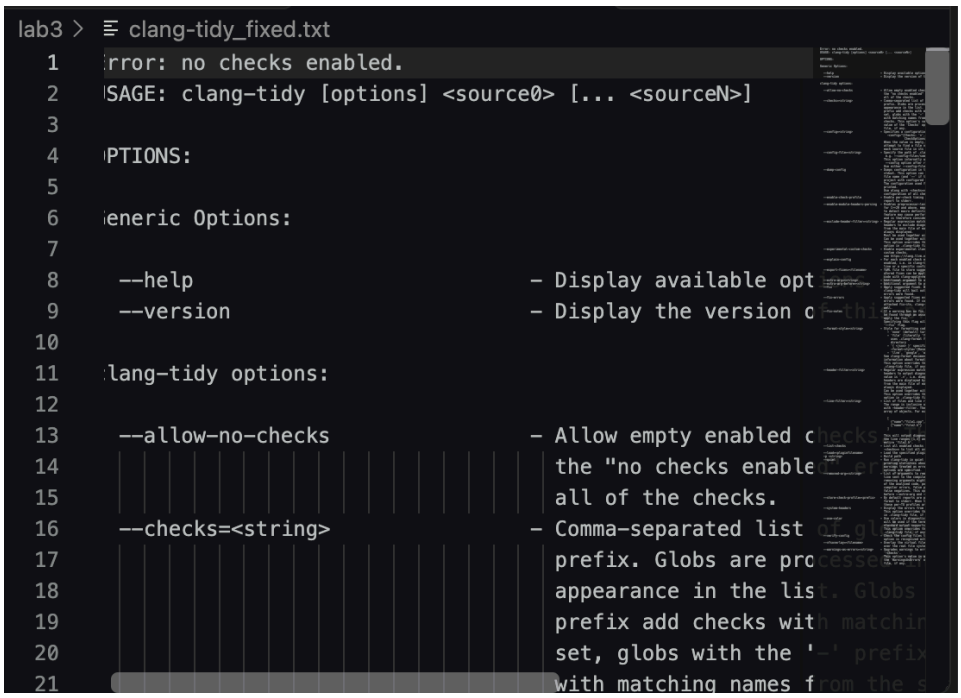
11)



12)



13)



14) Вот тут все особенно хорошо видно

15)

lab3 > flawfinder_fixed.txt

```
Flawfinder version 2.0.20, (C) 2001-2026 David A. Wheeler
Number of rules (primarily dangerous function names) in C
Examining okZoo.cpp

FINAL RESULTS:


ANALYSIS SUMMARY:

No hits found.
Lines analyzed = 49 in approximately 0.00 seconds (14769 lines/sec
Physical Source Lines of Code (SLOC) = 45
Hits@level = [0]   0 [1]   0 [2]   0 [3]   0 [4]   0 [5]   0
Hits@level+ = [0+]   0 [1+]   0 [2+]   0 [3+]   0 [4+]   0 [5+]
Hits/KSLOC@level+ = [0+]   0 [1+]   0 [2+]   0 [3+]   0 [4+]   0 [
Minimum risk level = 1

There may be other security vulnerabilities; review your code!
See 'Secure Programming HOWTO'
(https://dwheeler.com/secure-programs) for more information.
```

И теперь исправленные SCA (sbom_trivy_fixed.json, sbom_syft_fixed.json

lab3 > {} sbom_syft.json > ...

```
{"$schema":"http://cyclonedx.org/schema/bom-1.7.schema.js
```

)

16)

lab3 > {} sbom_trivy_fixed.json > ...


```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:33b336f0-f24a-450a-bfd0-22fd3
  "version": 1,
  "metadata": {
    "timestamp": "2026-10-19T09:43:57+00:00",
    "tools": {
      "components": [
        {
          "type": "application",
          "manufacturer": {
            "name": "Aqua Security Software Ltd."
          },
          "group": "aquasecurity",
          "name": "trivy",
          "version": "0.76.0"
        }
      ]
```

17)

```
● margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main) [1]> syft okZoo.cpp -o cyclonedx-json > sbom_syft_fixed.json
  ✔ Indexed file system                                                                                    okZoo.cpp
  ✔ Cataloged contents                                                   e1cc4ae4af81ce4361cb554c48bb6954cd8b95afee962f6ec2e55982315d3b5d
    ├── ✔ Packages                        [0 packages]
    └── ✔ Executables                     [0 executables]
```

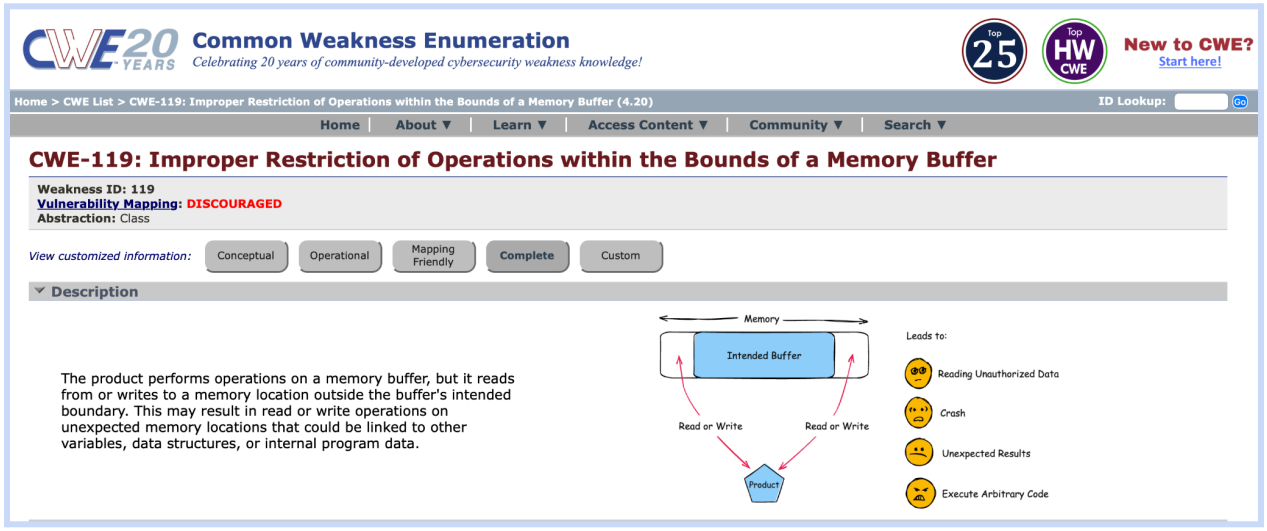
18)

```
● margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> trivy fs --scanners vuln --format cyclonedx . > sbom_trivy_fixed.json
  2026-10-19T12:43:57+03:00       INFO    [vuln] Vulnerability scanning is enabled
  2026-10-19T12:43:57+03:00       INFO    Number of language-specific files       num=0
● margaretlacoste@Margarets-MacBook-Air ~/w/S/lab3 (main)> pwd
  /Users/margaretlacoste/workspace/SDA/lab3
```

Что же нашли наши анализаторы cppcheck, clang-tidy и flawfinder?

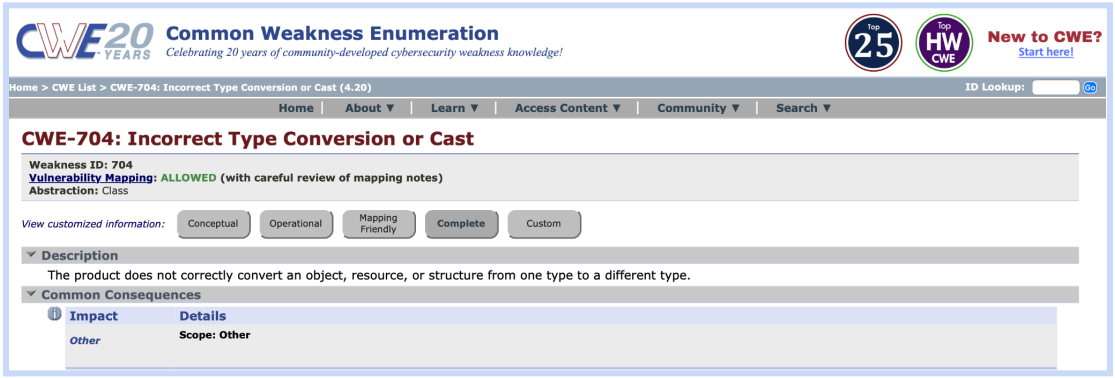
1) `<error id="invalidscanf" severity="warning" msg="sscanf() without field width limits can crash with huge input data."`

Это CWE-119 и TP(реальный риск)



2) `<error id="dangerousTypeCast" severity="warning" msg="Potentially invalid type conversion in old-style C cast, clarify/fix with C++ cast"`

Это CWE-704 и TP



3) <error id="constVariableReference" severity="style" msg="Variable 'a' can be declared as reference to const" verbose="Variable 'a' can be declared as reference to const" cwe="398"

Это CWE-398, CWE-134 и TP

CWE CATEGORY: 7PK - Code Quality

Category ID: 398
Vulnerability Mapping: PROHIBITED

Summary

This category represents one of the phyla in the Seven Pernicious Kingdoms vulnerability classification. It includes weaknesses that do not directly introduce a weakness or vulnerability, but indicate that the product has not been carefully developed or maintained. According to the authors of the Seven Pernicious Kingdoms, "Poor code quality leads to unpredictable behavior. From a user's perspective that often manifests itself as poor usability. For an adversary it provides an opportunity to stress the system in unexpected ways."

Membership

Nature	Type	ID	Name
MemberOf	V	700	Seven Pernicious Kingdoms
MemberOf	C	978	SFP Secondary Cluster: Implementation
HasMember	V	401	Missing Release of Memory after Effective Lifetime
HasMember	V	404	Improper Resource Shutdown or Release
HasMember	V	415	Double Free
HasMember	V	416	Use After Free
HasMember	V	457	Use of Uninitialized Variable
HasMember	B	474	Use of Function with Inconsistent Implementations
HasMember	B	475	Undefined Behavior for Input to API
HasMember	B	476	NULL Pointer Dereference
HasMember	B	477	Use of Obsolete Function

Vulnerability Mapping Notes

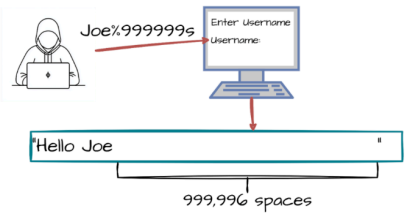
CWE-134: Use of Externally-Controlled Format String

Weakness ID: 134
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product uses a function that accepts a format string as an argument, but the format string originates from an external source.



Common Consequences

```
Examining vulnerableZoo.cpp <br>

<h2>Final Results</h2>
<ul>
<li>vulnerableZoo.cpp:17:5: <b>  [4] </b> (buffer) <i> strcpy:
 Does not check for buffer overflows when copying to destination [MS-banned]
 (<a href="https://cwe.mitre.org/data/definitions/120.html">CWE-120</a>).
 Consider using snprintf, strcpy_s, or strlcpy (warning: strncpy easily
 misused). </i></li>
<li>vulnerableZoo.cpp:18:5: <b>  [4] </b> (format) <i> printf:
 If format strings can be influenced by an attacker, they can be exploited
 (<a href="https://cwe.mitre.org/data/definitions/134.html">CWE-134</a>).
 Use a constant for the format specification. </i></li>
```

```
<li>vulnerableZoo.cpp:10:5: <b>  [2] </b> (buffer) <i> char:
 Statically-sized arrays can be improperly restricted, leading to potential
 overflows or other issues (<a
 href="https://cwe.mitre.org/data/definitions/119.html">CWE-119</a>!/<a
 href="https://cwe.mitre.org/data/definitions/120.html">CWE-120</a>).
 Perform bounds checking, use functions that limit length, or ensure that
 the size is larger than the maximum possible length. </i></li>
<li>vulnerableZoo.cpp:11:5: <b>  [2] </b> (buffer) <i> char:
 Statically-sized arrays can be improperly restricted, leading to potential
 overflows or other issues (<a
 href="https://cwe.mitre.org/data/definitions/119.html">CWE-119</a>!/<a
 href="https://cwe.mitre.org/data/definitions/120.html">CWE-120</a>).
 Perform bounds checking, use functions that limit length, or ensure that
 the size is larger than the maximum possible length. </i></li>
<li>vulnerableZoo.cpp:16:5: <b>  [2] </b> (buffer) <i> char:
 Statically-sized arrays can be improperly restricted, leading to potential
 overflows or other issues (<a
 href="https://cwe.mitre.org/data/definitions/119.html">CWE-119</a>!/<a
 href="https://cwe.mitre.org/data/definitions/120.html">CWE-120</a>).
 Perform bounds checking, use functions that limit length, or ensure that
 the size is larger than the maximum possible length. </i></li>
<li>vulnerableZoo.cpp:45:31: <b>  [1] </b> (buffer) <i> strlen:
 Does not handle strings that are not \0-terminated; if given one it may
 perform an over-read (it could cause a crash if unprotected) (<a
 href="https://cwe.mitre.org/data/definitions/126.html">CWE-126</a>). </i></li>
</ul>
<h2>Analysis Summary</h2>
```

CVE от sbom_trivy и sbom_syft

переполнение памяти в `deflate()` при сжатии специально сформированных данных

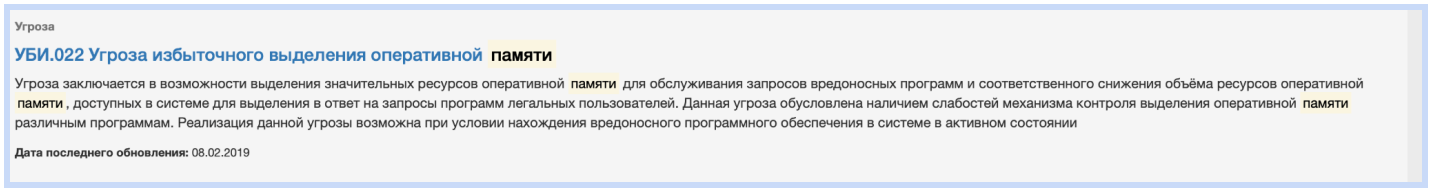
CVE-2018-25032

CWE-125 (Out-of-bounds Read)

OWASP - A06:2021

https://top10.owasp.org/2021/A06_2021-Vulnerable_and_Outdated_Components/index.html

БДУ ФСТЭК УБИ.022



Угроза

УБИ.022 Угроза избыточного выделения оперативной памяти

Угроза заключается в возможности выделения значительных ресурсов оперативной памяти для обслуживания запросов вредоносных программ и соответственного снижения объёма ресурсов оперативной памяти, доступных в системе для выделения в ответ на запросы программ легальных пользователей. Данная угроза обусловлена наличием слабостей механизма контроля выделения оперативной памяти различным программам. Реализация данной угрозы возможна при условии нахождения вредоносного программного обеспечения в системе в активном состоянии

Дата последнего обновления: 08.02.2019

heap overflow

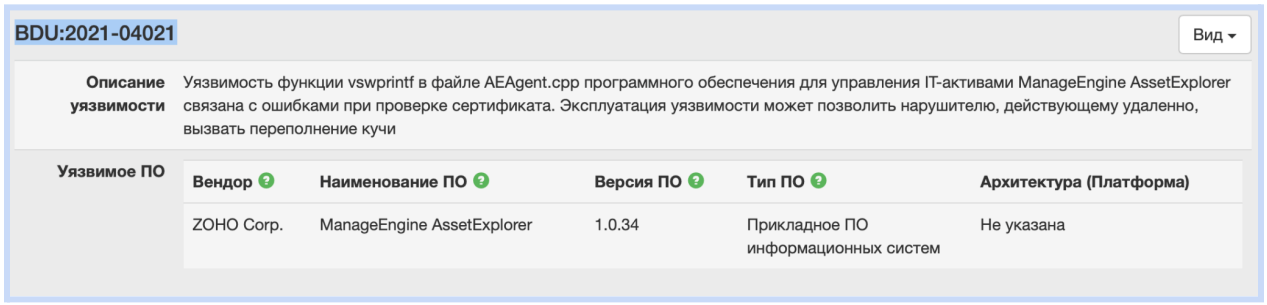
CVE-2022-37434

CWE-787

OWASP - A06:2021

https://top10.owasp.org/2021/A06_2021-Vulnerable_and_Outdated_Components/index.html

БДУ ФСТЭК: BDU:2021-04021

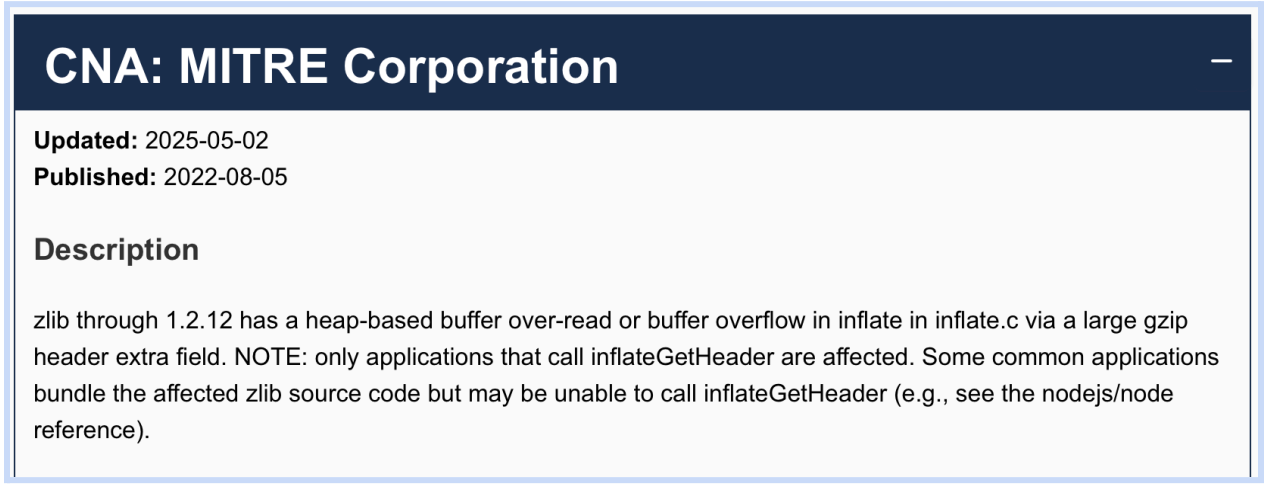


BDU:2021-04021 Вид

Описание уязвимости Уязвимость функции vswprintf в файле AEAgent.cpp программного обеспечения для управления IT-активами ManageEngine AssetExplorer связана с ошибками при проверке сертификата. Эксплуатация уязвимости может позволить нарушителю, действующему удаленно, вызвать переполнение кучи

Уязвимое ПО

Вендор	Наименование ПО	Версия ПО	Тип ПО	Архитектура (Платформа)
ZOHO Corp.	ManageEngine AssetExplorer	1.0.34	Прикладное ПО информационных систем	Не указана



CNA: MITRE Corporation

Updated: 2025-05-02
Published: 2022-08-05

Description

zlib through 1.2.12 has a heap-based buffer over-read or buffer overflow in inflate in inflate.c via a large gzip header extra field. NOTE: only applications that call inflateGetHeader are affected. Some common applications bundle the affected zlib source code but may be unable to call inflateGetHeader (e.g., see the nodejs/node reference).